



AUDITWARE

Slingshot Mobile Wallet - Security Audit

Performed by Auditware for Slingshot Finance

Date Completed: 9/30/2022

About Us

Auditware is a security auditing firm specializing in application and smart contract audits. We have also created Audit Wizard - a revolutionary web3 auditing tool. Request an audit and learn more at auditware.io.

Our Team

Indigo: Lead security engineer and primary auditor

Aidan: Engineer and assistant auditor

Contact

twitter.com/auditware_

auditware.eth

Table of Contents

1. Overview
2. Findings
 - a. AW-SW-1 - MMKV uses AES-CFB-128 to encrypt data at rest without integrity validation
 - b. AW-SW-2 - PIN can be brute-forced
 - c. AW-SW-3 - Transaction auth disablement without re-auth
 - d. AW-SW-4 - Log injection
 - e. AW-SW-5 - Endpoint defense-in-depth
 - f. AW-SW-6 - Potential entry of sensitive user information
 - g. AW-SW-7 - Unbounded wallet names allowed
3. Focus Areas
 - a. Wallet management
 - i. Key generation
 - ii. Encryption
 - iii. Integrity assurance
 - b. Authentication
 - i. Bypass
 - ii. Brute force capability
 - c. Data tampering and UI redress

Overview

The Slingshot Mobile Wallet is an intricately designed non-custodial wallet that allows users to buy, sell, swap and store crypto, and view real-time data and price charts, with full control over their private keys and crypto. The focus of this audit was evaluating potential security risks around wallet management, cryptography configuration, local authentication, and transaction parameter tampering. The core of the app security lies in the protection of the wallet private keys and mnemonics, and the ability to sign authorized transactions on behalf of the user.

The results of our audit have concluded that there are no major security concerns with the Slingshot Mobile Wallet. Only a few minor improvements are suggested to bolster security around authentication and encryption attacks. Below you can find a list of our findings and a detailed description of our focus areas.

Findings

AW-SW-1 - MMKV uses AES-CFB-128 to encrypt data at rest without integrity validation

Severity: *Medium*

Code: WalletStore.ts:102-103

```
this.storage.set(WALLET_MANAGER_KEY, JSON.stringify(wallet))
this.storage.recrypt(encryptionKey)
```

Description:

- MMKV uses AES-CFB-128 to encrypt data at rest, but integrity validation is not present.
- The `recrypt()` call is redundant, as encryption is performed during storage.

Recommendation: Use an appropriate encryption library with an authenticated encryption mode or integrity verification to protect the data before it is stored.

AW-SW-2 - PIN can be brute-forced

Severity: *Medium*

Code: useUnlock.ts, useLocalAuthentication.ts:73

```
return pinStore.verifyPin(pin)
```

Constants.ts:30

```
export const PIN_LENGTH = 6
```

Description:

- There is no lockout mechanism upon multiple failed PIN unlock attempts.

- PINs are fixed to six digits, meaning there are 10^6 possible PIN combinations, which is feasibly brute forceable by a determined attacker.
 - PINs six digits in length may not support users who have stricter security requirements.
- Users may set insecure PINs ('111111', '12345', etc.).

Recommendation: Add a timed lockout to the PIN authentication flow to prevent brute-force attempts. Allow users to set PINs larger than six digits. It is also recommended to inform users to set a strong passcode to protect their wallet.

AW-SW-3 - Transaction auth disablement without re-auth - FIXED

Severity: *Low*

Code: SettingsScreen.tsx

Description:

- PIN/biometrics auth for transactions can be disabled in settings without requiring re-authentication of that method.

Recommendation: ~~Require re-authentication before disabling this setting.~~ This has been fixed at the time of writing this report.

AW-SW-4 - Log injection

Severity: *Very Low*

Code: useHandleLinks.ts:105

```
Logger.log('🔗', 'Unhandled url: ${rawURL}')
```

useUpdateWallet.ts:87

```
analytics.track('update_wallet', {...})
```

Description:

- Log injection via user input logging of URLs and analytics tracking of wallet names is possible.
- The impact of log injection is usually very low. However there is the potential for attackers to forge log messages or inject malicious code if logs are viewed in a context vulnerable to script injection.

Recommendation: Sanitize or encode user input in log messages.

AW-SW-5 - Endpoint defense-in-depth

Severity: *Very Low*

Description:

- The Slingshot API endpoint is potentially vulnerable to network attacks enabled by compromised public certificate authorities. The Slingshot API is only used for low-trust read operations and does not present a great risk.
- TLS certificates may be manually pinned (e.g. via a library such as [react-native-ssl-pinning](#)) to prevent such attacks.

Recommendation: It is recommended to pin the certificates of the Slingshot API and any applicable external dependencies.

AW-SW-6 - Potential entry of sensitive user information - [FIXED](#)

Severity: *Informational*

Code: UserFeedback.tsx

Description:

- Users could potentially enter sensitive information in free-form text entries for bug reports and feature requests (e.g. mnemonics or passcodes).

Recommendation: ~~Alert the user not to submit any sensitive data.~~ This has been fixed at the time of writing this report.

AW-SW-7 - Unbounded wallet names allowed

Severity: *Informational*

Code: useUpdateWallet.ts:90,159

Description:

- Users may specify wallet names of unbounded length. Large enough wallet names prevent certain UI flows from rendering, making it impossible to delete such a wallet without reinstalling the app.
- This is not necessarily a security risk, but wallet names are stored in the analytics server and could potentially contribute to reduced availability or increased infrastructure costs.

Recommendation: Enforce a maximum wallet name length.

Focus Areas

The communication with the backend, authentication, and authorization for Firebase operations, and everything related to user preferences was out of scope of this audit.

Wallet management

- Key generation
 - Private keys for wallets are securely generated using a suitable source of randomness, using an appropriate library: [crypto](#).
- Encryption

- Encryption is performed automatically, by the MMKV library, with a master private key stored in the local secure storage of the device. MMKV performs this encryption using AES in CFB-128 mode, which is generally adequate for assuring the confidentiality of data.
- Using MMKV for encryption presents a dependency risk, as MMKV was not designed under adequate scrutiny as a cryptography library and has not been sufficiently battle-tested with regard to encryption.
- These and other potential unknown issues lead us to suggest performing encryption separate to storage and using a dedicated, well-vetted library.
- Integrity assurance
 - There is no integrity verification performed on encrypted data.
 - MMKV does not provide any supplemental or built-in integrity validation as part of the encryption process.
 - It is recommended that encrypted data be stored alongside generated HMACs (to be verified during decryption) or that an authenticated encryption cipher, such as AES-GCM, be used.

Authentication

- Bypass
 - No feasible method for bypass of the PIN or biometrics was found. The PIN lock screen uses a custom entry mechanism that protects against overloading and malformed inputs, which are common vulnerabilities of PIN/passcode locks.
- Brute-force capability
 - The unlock PIN is fixed to six digits. It is recommended to allow users to set longer PINs.
 - PIN codes have no complexity requirements. It is advised to prevent users from using insecure PINs (e.g. '111111', '123456', etc.) or to at least add an advisory to select a secure PIN to the PIN creation screen.
 - There is no brute-force protection. An attacker could guess a user's PIN with enough time (10^6 combinations is feasible for a determined attacker). It is recommended to add a timed lockout on repeated tries to unlock the wallet.

Data tampering and UI redress

- The data sources for the UI views in the app include the Slingshot backend, Firebase database, and the ethers provider.
 - Transaction data is gathered from trusted sources but is also ultimately verified by the user before confirming transactions. Some inputs, such as QR code and WalletConnect links, can be risky for users, which is why it is important that users verify transaction parameters before submitting transactions. It is recommended to convey this risk to users with an advisory during onboarding and/or upon transacting.
- The ethers provider is used to estimate gas limits for transactions but it is resistant to tampering as the default provider uses a quorum of data sources to reach a consensus on data values.
- The UI is properly architected, and no avenues for redress were discovered.